1. Introduction and Objectives

The objective of this project was to implement and compare two fundamental Reinforcement Learning (RL) algorithms—**Classic Q-Learning** and **Deep Q-Network (DQN)**—to solve a navigation task in a grid-based maze environment. The study evaluates the impact of initialization strategies, exploration policies, and reward structures on the agent's ability to reach a goal while minimizing steps.

2. Methodology and Implementation

2.1 RL Framework

- **State Space:** Discrete grid coordinates (x, y). In DQN, these are normalized to a [0, 1] range.
- **Action Space:** Discrete (Up, Down, Left, Right).
- **Environment:** A 10 x 10 maze with static obstacles, a starting point at (1, 1), and a goal at (10, 10). [1-indexed]

2.2 Algorithms Evaluated

- **Classic Q-Learning:** Utilizes a tabular approach (Q-table) to store state-action values.
- **Deep Q-Learning (DQN):** Utilizes a Multi-Layer Perceptron (MLP) with three fully connected layers to approximate the Q-function, employing a Replay Buffer and a Target Network for stability.

2.3 Experimental Variables

We tested 24 baseline configurations based on the following permutations:

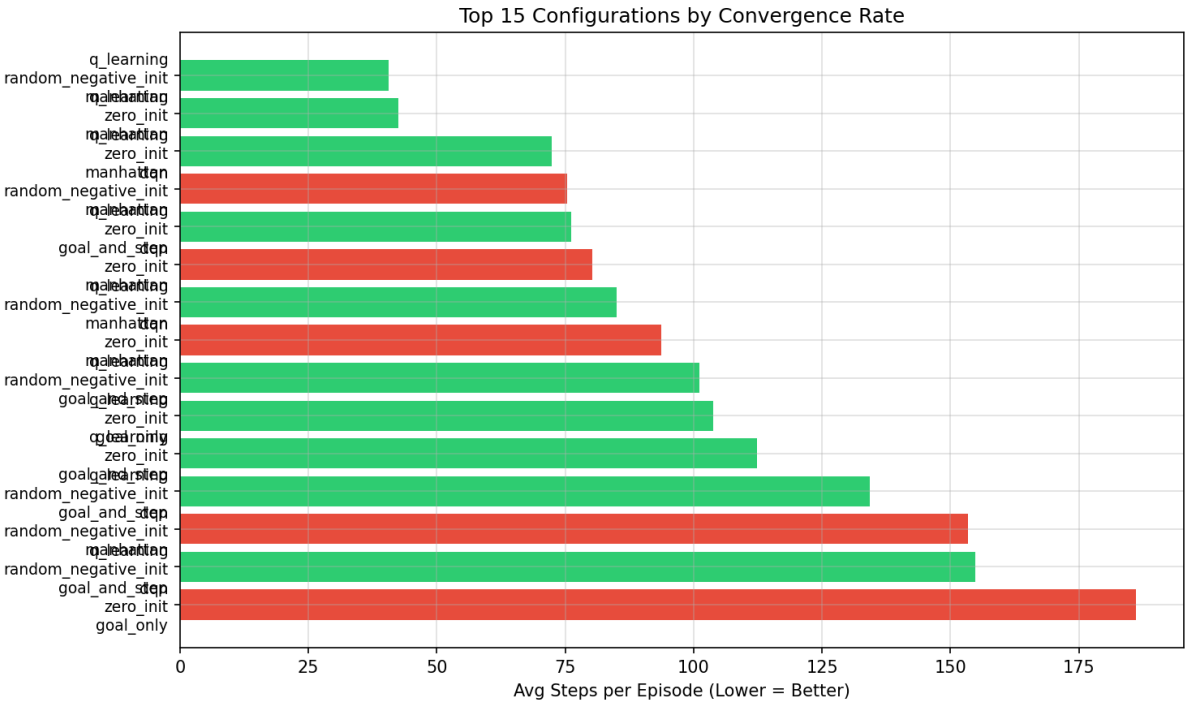
1. **Algorithms:** Q-Learning vs. DQN.
2. **Initializations:** Zero vs. Random Negative.
3. **Exploration Policies:** Epsilon-Decay vs. Performance-Based.
4. **Reward Modes:** Goal Only, Goal and Step, and Manhattan Distance.

3. Results and Performance Comparison

3.1 Evaluation Metrics

To determine which configuration is "better" beyond simply reaching the goal, we utilize four primary metrics:

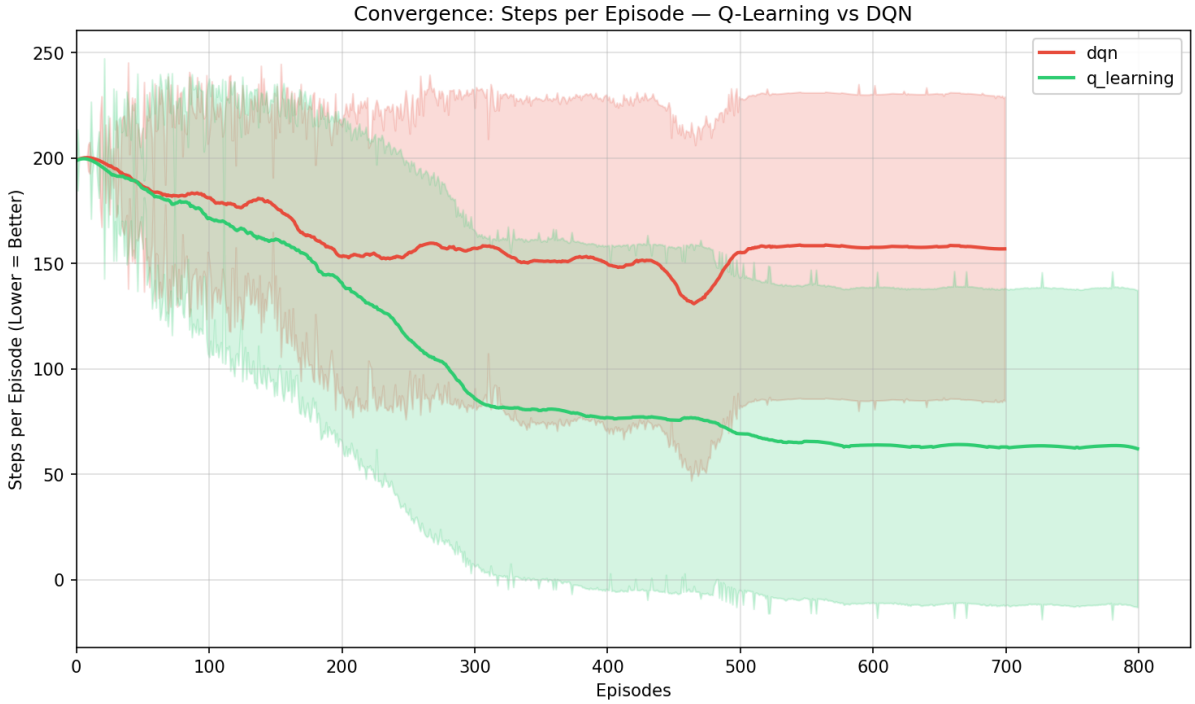
- **Convergence Rate (Steps per Episode):** We measure how many episodes it takes for the "Steps per Episode" curve to flatten. A faster drop indicates a more efficient learning strategy. Also since number of steps for the optimal solutions is considerably less than the Max_Steps, lower count implies nearing optimal solution.
- **Average Reward (Batch Metric):** The mean reward accumulated over the final 100 episodes. This indicates the stability and optimality of the learned path.
- **Wall-Clock Time (Efficiency):** The total computational time required to complete the training episodes, which highlights the overhead differences between Tabular and Deep methods.



3.2 DQN vs. Classic Q-Learning

The results indicate that **Classic Q-Learning performed better** in this specific 10 X 10 maze environment.

- **Performance:** Classic Q-Learning achieved the better convergence to minimized Steps per episode.
- **Circumstances:** Q-Learning excelled because the state space is small and discrete, allowing the Q-table to update exact values quickly. DQN, while powerful, introduced approximation noise and required more tuning of the replay buffer and target network to reach similar stability.
- **Observation:** DQN was more consistent in its "Time per Run," whereas Q-Learning's speed was highly dependent on the exploration policy's efficiency.

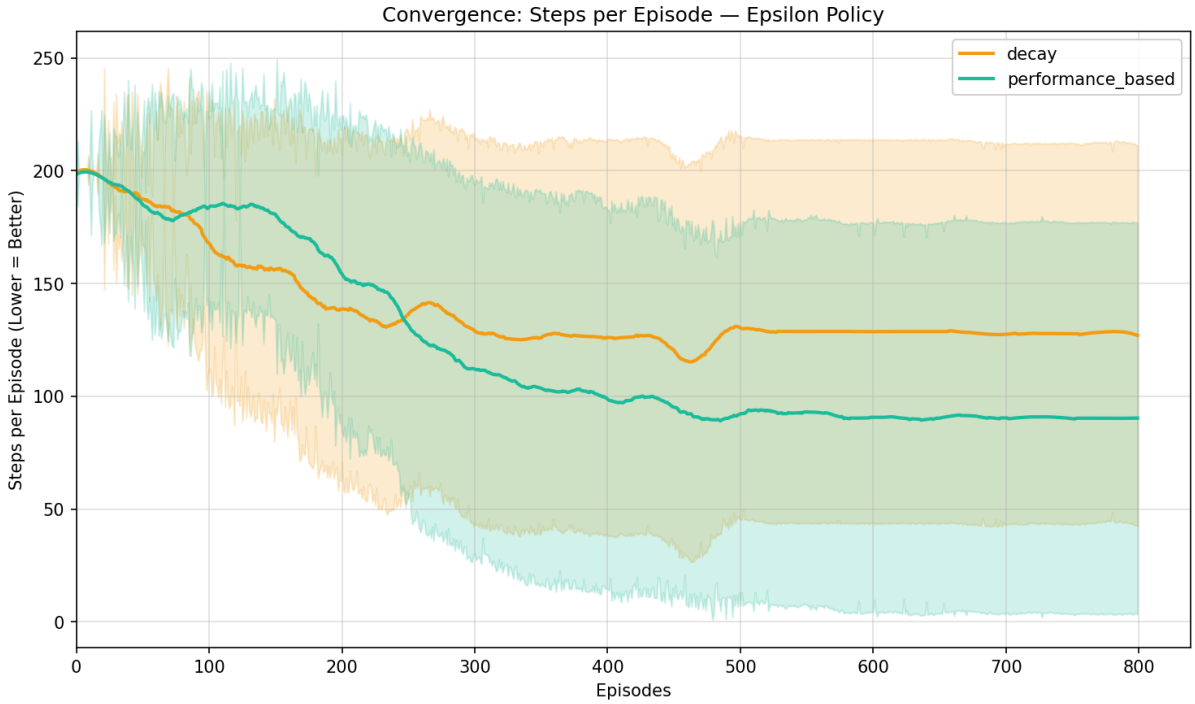


3.3 Exploration Versus Exploitation

We compared standard **Epsilon Decay** against a **Performance-Based** strategy.

- **Q-Learning Trends:** Performance-based exploration reached the #1 spot. By only decaying epsilon when the agent showed improvement, it ensured sufficient exploration in early "stuck" phases.

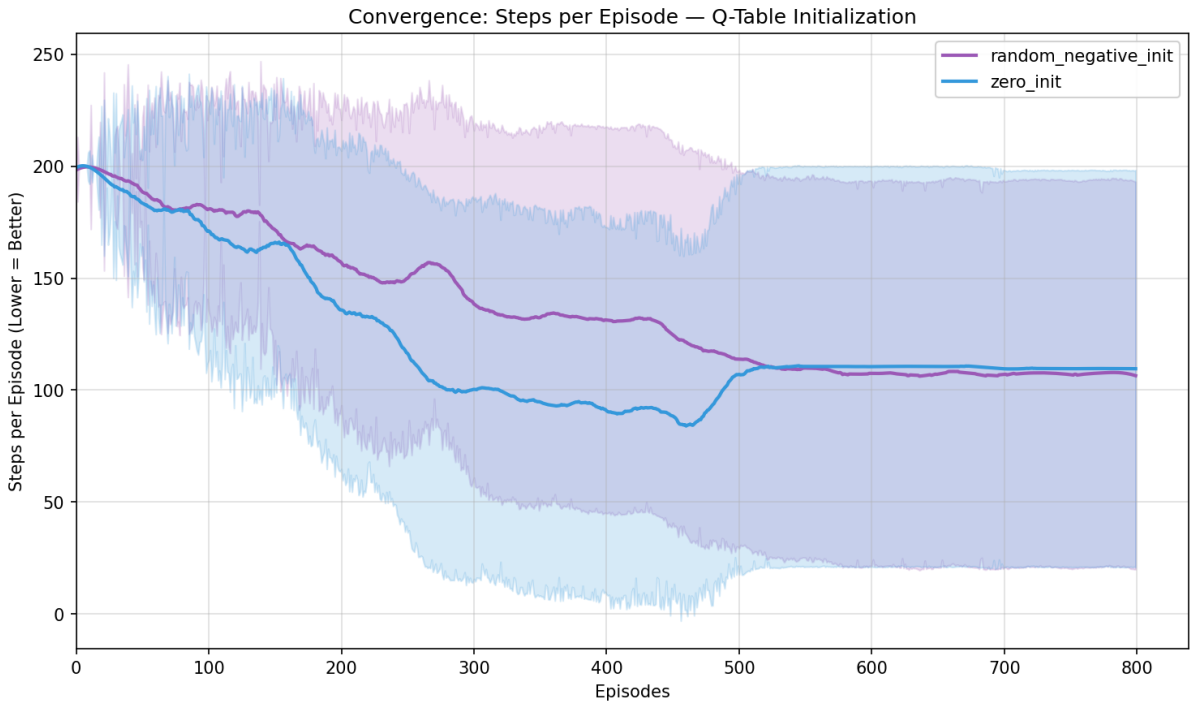
- **DQN Trends:** DQN was more sensitive to the policy. Under Epsilon Decay, DQN often converged to sub-optimal paths because it stopped exploring before the neural network could generalize the entire maze.
- **Comparison:** Across both algorithms, the Performance-Based policy resulted in lower steps per episode, implying it converged faster to optimal solution



3.4 Initialization Strategies

We evaluated **All-Zeros** vs. **Small Random Negative** initialization.

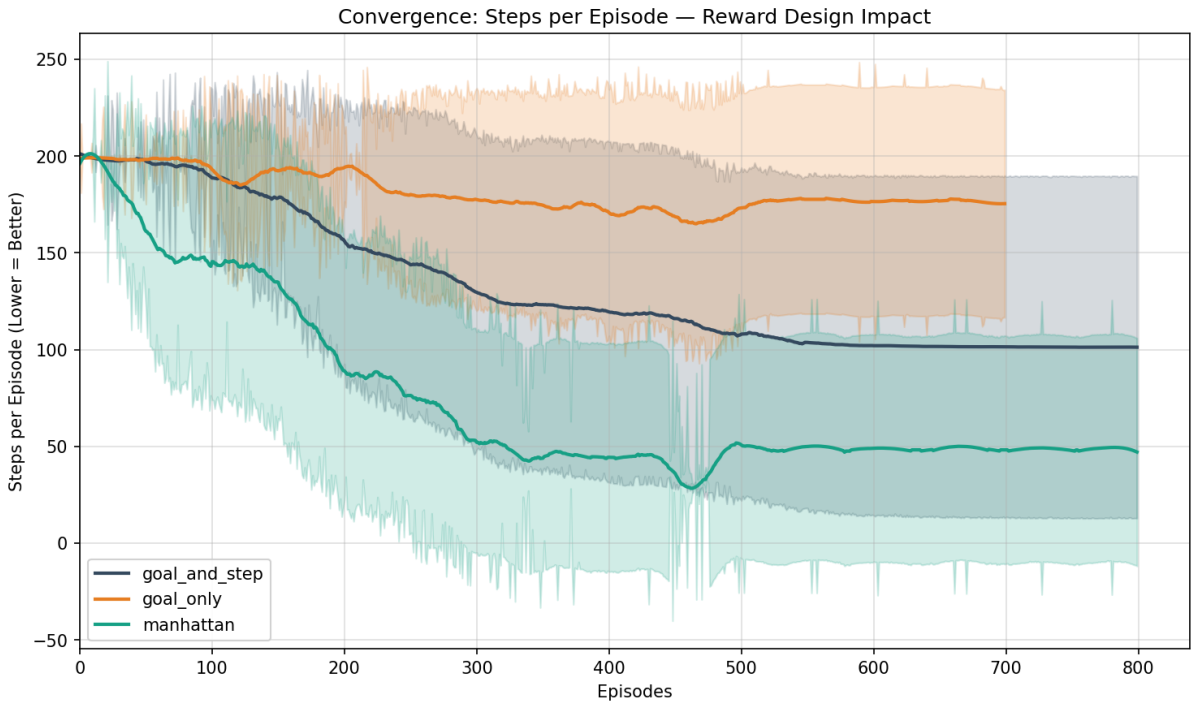
- **Exploration Behavior:** Random negative initialization encouraged more diverse early exploration (as seen in the "random" look of early heatmaps) because the agent was less likely to repeat actions that yielded an initial negative value.
- **Convergence Rate:** All-Zeros initialization converged slightly faster in Times for Q-Learning but reached nearly identical steps per episode.
- **Final Performance:** Initialization had the smallest impact on the steps per episode compared to other variables, suggesting both algorithms are robust to starting weights if the reward signal is strong enough.



3.5 Reward Design

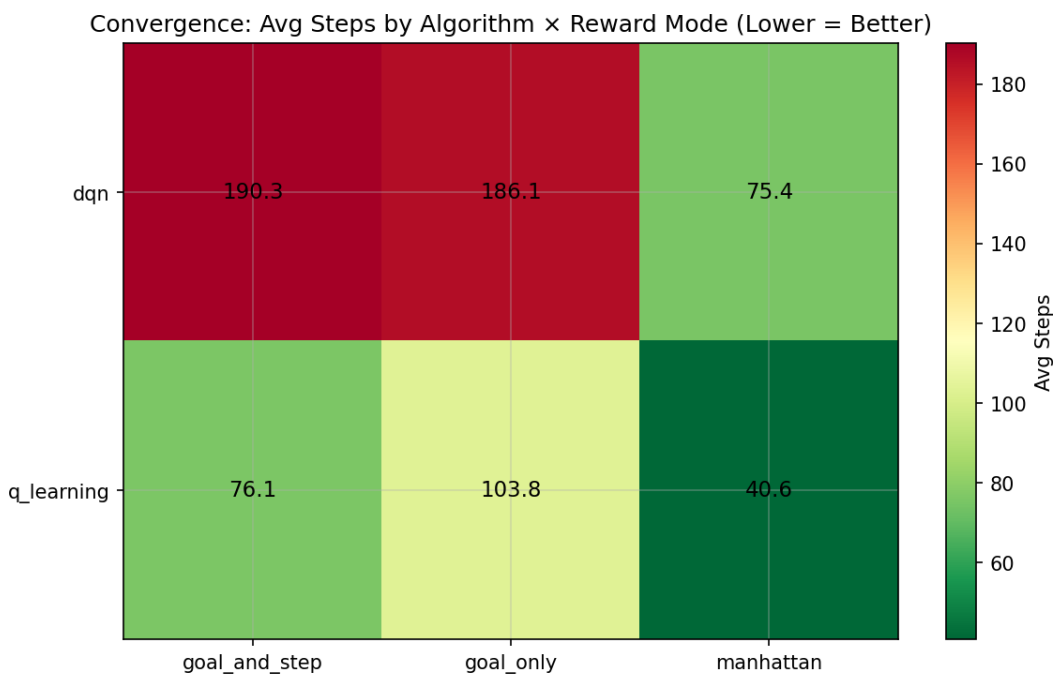
This was the most influential factor in learning dynamics.

- **Manhattan-Shaped (Best):** This design led to the fastest convergence and highest stability. By providing a gradient toward the goal, it removed the "blind" search phase, leading to Average Steps as low as **40.6**.
- **Goal + Step Penalty:** While this encourages shorter paths, it often led to negative average rewards (down to **-1.99**) if the agent didn't find the goal quickly. The penalty often "overwhelmed" the agent, causing it to stall.
- **Goal Only:** This sparse reward resulted in the slowest convergence. Without feedback during the journey, the agent's success was largely based on random chance until a path was eventually found and backpropagated.



3.6 Path Optimization Visual

The effectiveness of the combined "Winner" configuration (Q-Learning + Manhattan + Performance-Based) is best seen in the transition of the agent's trajectory.



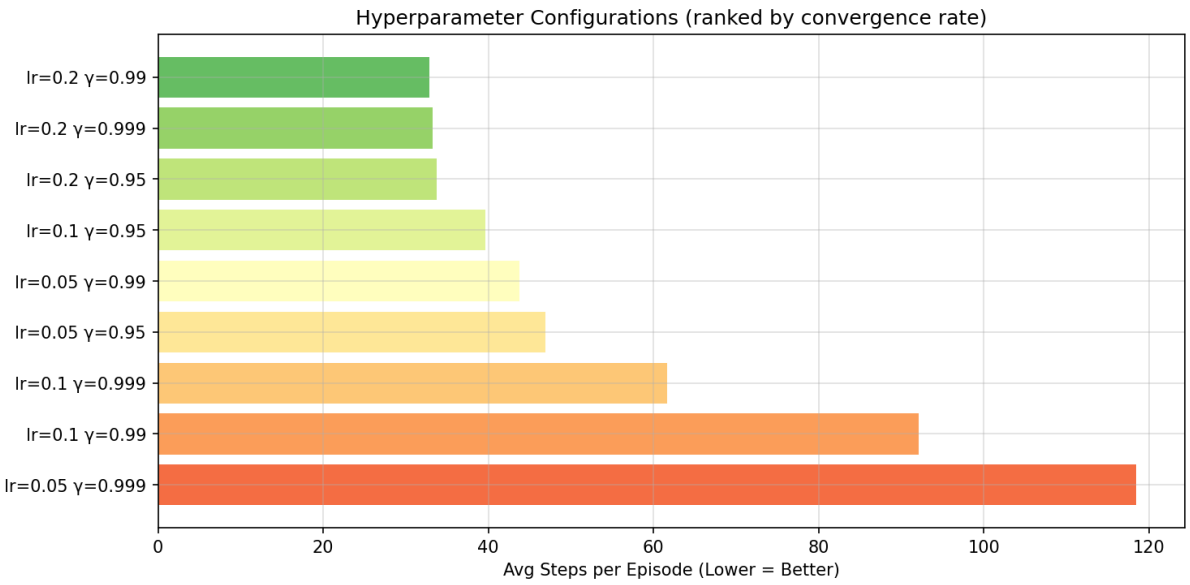
The heatmap confirms the transition from high-entropy exploration (wide-spread visits) to a deterministic, optimized path (narrow, bright line to goal).

3.6 Hyperparameter Tuning (alpha and gamma)

For the best-performing baseline configuration (Q-Learning, Manhattan, Performance-Based, Random Negative Init), we conducted a grid search across three Learning Rates (alpha) and three Discount Factors (gamma).

- **Learning Rate (alpha):** Higher learning rates (alpha=0.2) led to significantly faster convergence, with the agent reaching the goal in an average of **32.9 steps** compared to over 46 steps at alpha=0.05. While high learning rates can sometimes cause instability, the discrete nature of the 10 times 10 maze allowed the agent to lock into optimal Q-values without oscillation.

- **Discount Factor (gamma):** The agent was sensitive to the discount factor. Mid-range values (gamma=0.99) consistently provided the best results, as they balanced immediate rewards with long-term goals. Extreme values (e.g., gamma=0.999) occasionally led to slower convergence (Avg Steps: 118.5) at lower learning rates, likely due to the "over-propagation" of future rewards in a small state space.

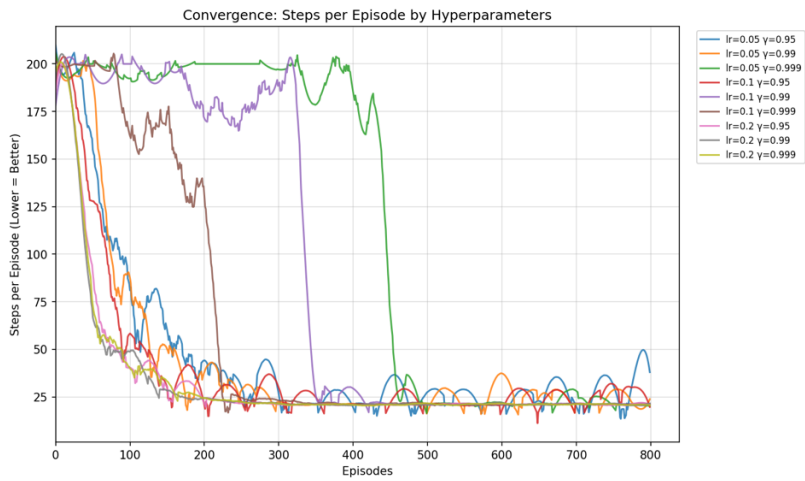


The leaderboard illustrates that the combination of alpha=0.2 and gamma=0.99 yielded the lowest steps per episode.

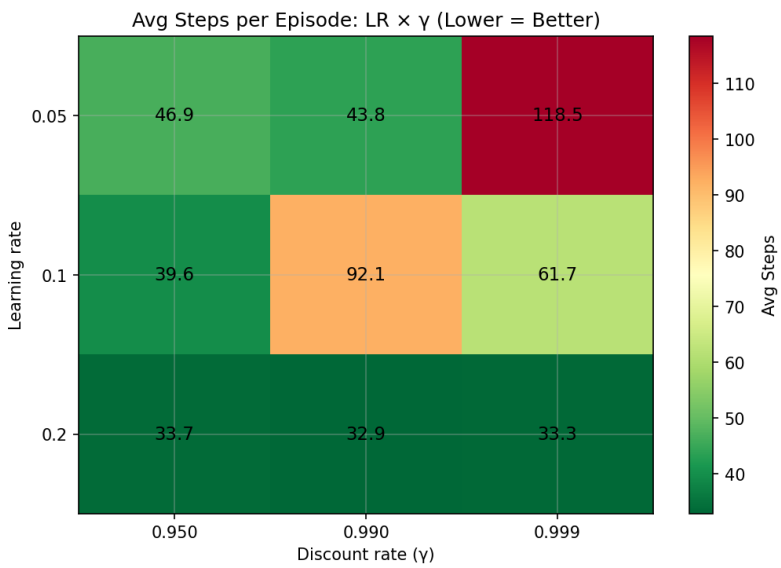
3.7 Visual Analysis of Training and Trajectories

To validate the agent's behaviour, we examined the internal metrics and spatial movement patterns during training.

Learning Curves: The aggregated learning curves show that higher learning rates result in a much steeper steps descent, reaching a low point before the mid-point of training.



Trajectory Heatmaps: Early in training, the heatmap shows high entropy, with the agent visiting most of the maze cells. As learning progresses, the visit frequency concentrates into a specific, optimal "highway" from (1,1) to (10,10), proving that the agent is not just reaching the goal, but is optimizing the path.



3.8 What Makes a Good Learning Algorithm?

Reflecting on the 30 experiments conducted, we can define a "superior" algorithm through three specific lenses beyond just the final score:

1. **Convergence Speed:** A good algorithm should reach an acceptable solution quickly. In our tests, **Q-Learning with Manhattan rewards** was superior because it required significantly fewer episodes to reach a success rate of 100% compared to DQN or "Goal-Only" reward structures.
2. **Stability:** Stability is marked by low variance in rewards once the agent has learned the task. The **Performance-Based** exploration policy proved more stable than simple decay because it prevented the agent from "unlearning" the path during late-stage random exploration.
3. **Generalization and Robustness:** While Q-Learning was faster for this 10 times 10 grid, **DQN** theoretically offers better generalization for larger, continuous spaces. However, in this discrete task, the "goodness" of the algorithm was defined by the **Reward Design**. An algorithm is only as good as the signal it receives; Manhattan-shaping turned a difficult search problem into a guided optimization task.

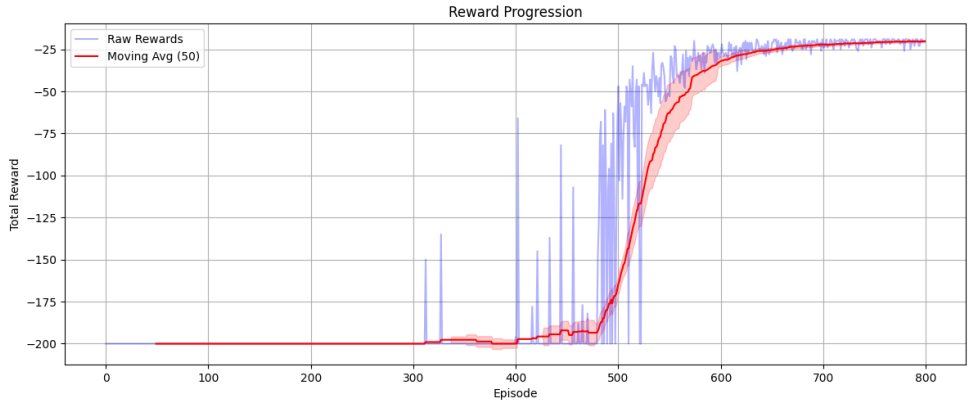
Final Best Result: * Config: Q-Learning | alpha=0.2 | gamma=0.99 | Manhattan | Performance-Based | Random Neg Init.

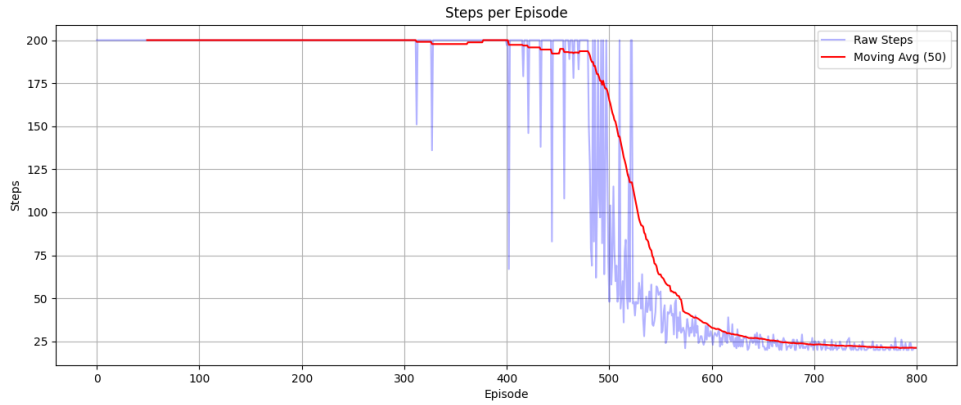
3.9 Extra Credit attempt

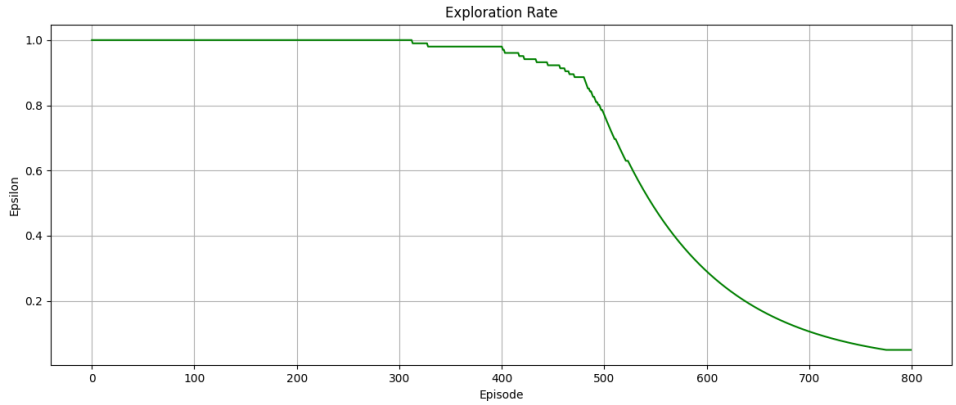
. Behavior Analysis: "Goal 1 | Step -0.01" (training_metrics.png)

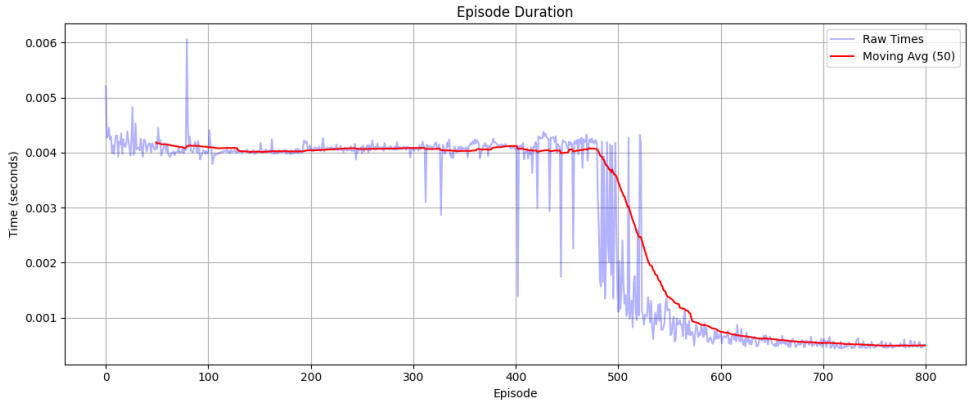
In this configuration, the agent is primarily motivated by the **+1.0 reward** at the finish line. Because the step penalty is negligible (-0.01), the agent is "forgiven" for exploration.

- **Trajectory:** The agent often takes longer paths during the early and middle phases of training because the cost of an extra 10 steps is only -0.1, which doesn't significantly diminish the +1.0 glory of reaching the goal.
- **Learning Curve:** The reward curve climbs steadily toward the 1.0 mark. It is a "low-stress" environment where the agent quickly learns *that* it should reach the goal, but takes its time optimizing the *how*.


Reward Progression
Raw Rewards
Moving Avg (50)
Total Reward
Episode

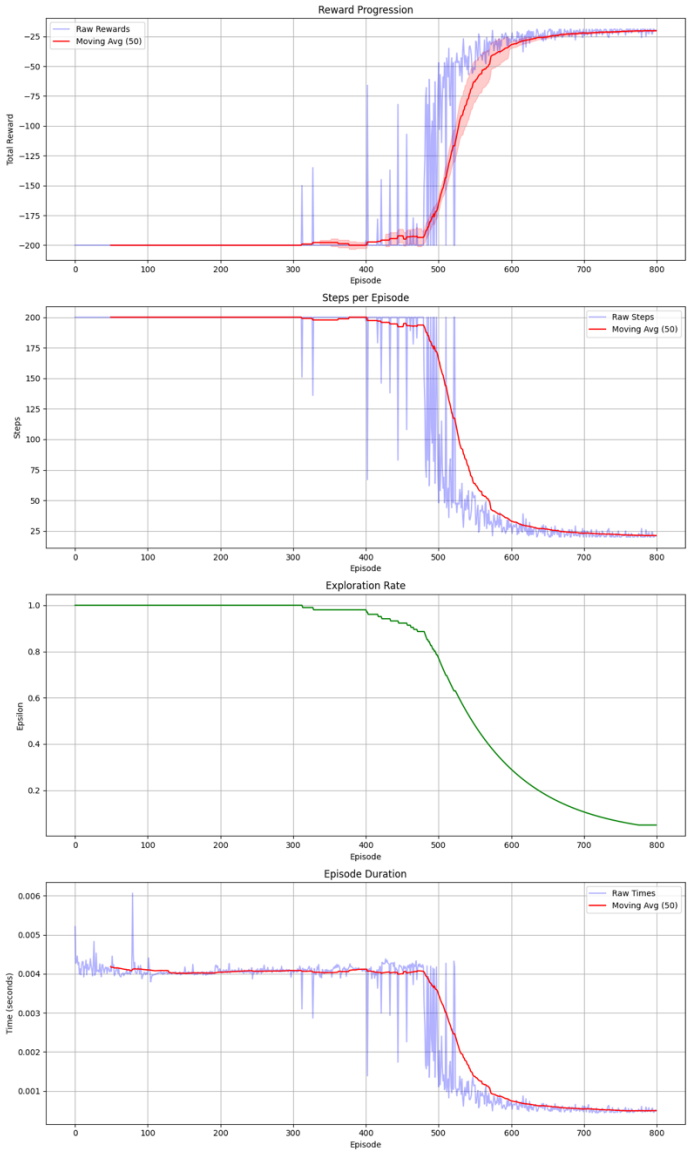

Steps per Episode
Raw Steps
Moving Avg (50)
Steps
Episode


Exploration Rate
Epsilon
Episode


Episode Duration
Raw Times
Moving Avg (50)
Time (seconds)
Episode

2. Behavior Analysis: "Goal 0 | Step -1" (training_metrics2.png)

This configuration represents a "survival" or "time-debt" model. Reaching the goal provides no points (0); instead, the agent is "taxed" **-1.0 for every second** it remains in the maze.

Trajectory: This creates an incredibly aggressive drive for efficiency. An extra 10 steps costs the agent -10.0 points—ten times the magnitude of the standard goal reward in the other config.

Learning Curve: The reward curves stay entirely in the negative "debt" zone. Success is measured by how close to zero the agent can get. You will notice a much steeper drop-off in the **Steps per Episode** plot, as the agent is brutally punished for any backtracking or standing still.